- Result: **False** (can_read is `False`)

Conclusion

Continue applying this structured analysis for the remaining expressions in the PDF. For each expression, identify the logical flow, determine where short-circuiting occurs, and evaluate the final result step-by-step. This method will enhance your understanding of logical expressions and short-circuit evaluation in programming contexts.

Share Rewrite

follow up till all question

Sources

Set of 20 Logical Expressions (Short-Circuit Evaluation)
File · 1

Answer

Here's a detailed analysis and evaluation of all 20 logical expressions from the PDF, focusing on the logical flow, short-circuit evaluation, and step-by-step reasoning for each scenario.

1. Checking Valid User Input

Expression:

```python
is_valid = (username == "admin" and len(password) >= 4) or (len(username) < 5)
```

- **Logical Flow**: Validates user input based on username and password.
- **Short-Circuit Evaluation**: If `username == "admin"` is `False`, the second condition `(len(username) < 5)` is still evaluated because of the `or`.
- **Step-by-Step Evaluation**:
  - If `username` is "admin" and `password` is "1234": **True**
  - If `username` is "user" and `password` is "123": **True** (evaluates second condition).

2. Resource Allocation Check

Expression:

```python
allocate_resource = (cpu_available > 50 and memory_available > 2000) or memory_available == 4096
```

- **Logical Flow**: Checks CPU and memory availability.
- **Short-Circuit Evaluation**: If `cpu_available > 50` is `False`, then `memory_available == 4096` is evaluated.
- **Step-by-Step Evaluation**:
  - If `cpu_available` is 60 and `memory_available` is 2048: **True**
  - If `cpu_available` is 40 and `memory_available` is 2048: **False** (evaluates second condition).

3. Login Attempt Validation

Expression:

```python
login_success = (attempts < 5 or not locked) and (attempts > 0)
```

- **Logical Flow**: Validates if a login attempt can proceed.
- **Short-Circuit Evaluation**: If `attempts < 5` is `True`, `not locked` is not evaluated.
- **Step-by-Step Evaluation**:
  - If `attempts` is 3 and `locked` is `False`: **True**
  - If `attempts` is 5 and `locked` is `True`: **False** (evaluates both conditions).

4. Product Availability Check

Expression:

```python
can_purchase = (in_stock > 5 or on_hold < 5) and in_stock > 0
```

- **Logical Flow**: Determines if a product can be purchased.

- **Short-Circuit Evaluation**: If `in_stock > 5` is `True`, `on_hold < 5` is not evaluated.
- **Step-by-Step Evaluation**:
  - If `in_stock` is 15 and `on_hold` is 10: **True**
  - If `in_stock` is 3 and `on_hold` is 6: **False** (evaluates both conditions).

5. File Operation Check

Expression:

```python
can_read = (file_exists and not file_opened) or file_opened
```

- **Logical Flow**: Checks if a file can be read.
- **Short-Circuit Evaluation**: If `file_exists` is `False`, `not file_opened` is not evaluated.
- **Step-by-Step Evaluation**:
  - If `file_exists` is `True` and `file_opened` is `False`: **True**
  - If `file_exists` is `True` and `file_opened` is `True`: **True**
  - If `file_exists` is `False`: **False**.

6. Discount Eligibility

Expression:

```python
eligible_for_discount = (is_member and purchase_amount >= 100) or purchase_amount > 150
```

- **Logical Flow**: Determines discount eligibility based on membership and purchase amount.
- **Short-Circuit Evaluation**: If `is_member` is `True`, check `purchase_amount >= 100`.
- **Step-by-Step Evaluation**:
  - If `is_member` is `True` and `purchase_amount` is 120: **True**
  - If `is_member` is `False` and `purchase_amount` is 160: **True**.

7. System Update Condition

Expression:

```python
can_update = (is_online and battery_level > 40) or battery_level > 80
```

- **Logical Flow**: Checks if the system can be updated.
- **Short-Circuit Evaluation**: If `is_online` is `False`, `battery_level > 40` is not evaluated.
- **Step-by-Step Evaluation**:
  - If `is_online` is `True` and `battery_level` is 50: **True**
  - If `is_online` is `False` and `battery_level` is 90: **True**.

8. Travel Eligibility Check

Expression:

```python
travel_allowed = (has_passport and has_visa) or not has_visa
```

- **Logical Flow**: Checks travel eligibility based on passport and visa.
- **Short-Circuit Evaluation**: If `has_passport` is `True`, check `has_visa`.
- **Step-by-Step Evaluation**:
  - If `has_passport` is `True` and `has_visa` is `True`: **True**
  - If `has_passport` is `True` and `has_visa` is `False`: **True**.
  - If `has_passport` is `False`: **True**.

9. Exam Eligibility

Expression:

```python
exam_eligible = (attendance >= 75 and assignments_submitted) or attendance > 80
```

- **Logical Flow**: Determines if a student can take the exam.

- **Short-Circuit Evaluation**: If `attendance >= 75` is `False`, `assignments_submitted` is not evaluated.
- **Step-by-Step Evaluation**:
  - If `attendance` is 80 and `assignments_submitted` is `True`: **True**
  - If `attendance` is 70 and `assignments_submitted` is `False`: **False**.

10. Security Check for File Access

Expression:

```python
access_file = (has_permission or is_owner) and not has_permission
```

- **Logical Flow**: Checks if a user can access a file.
- **Short-Circuit Evaluation**: If `has_permission` is `True`, `not has_permission` is not evaluated.
- **Step-by-Step Evaluation**:
  - If `has_permission` is `False` and `is_owner` is `True`: **True**
  - If `has_permission` is `True`: **False**.

11. Online Purchase Approval

Expression:

```python
purchase_successful = (payment_processed and stock_remaining > 0) or stock_remaining > 5
```

- **Logical Flow**: Validates if an online purchase can proceed.
- **Short-Circuit Evaluation**: If `payment_processed` is `False`, `stock_remaining > 0` is not evaluated.
- **Step-by-Step Evaluation**:
  - If `payment_processed` is `True` and `stock_remaining` is 3: **True**
  - If `payment_processed` is `False` and `stock_remaining` is 6: **True**.

12. Application Form Validation

Expression:

```python
form_accepted = (is_complete and has_photo) or not is_complete
```

- **Logical Flow**: Checks if an application form can be accepted.
- **Short-Circuit Evaluation**: If `is_complete` is `False`, `has_photo` is not evaluated.
- **Step-by-Step Evaluation**:
  - If `is_complete` is `True` and `has_photo` is `True`: **True**
  - If `is_complete` is `False`: **True**.

13. Server Health Monitoring

Expression:

```python
server_healthy = (cpu_usage < 80 or memory_usage < 85) and (cpu_usage < 90 or memory_usage < 95)
```

- **Logical Flow**: Checks if the server is healthy based on resource usage.
- **Short-Circuit Evaluation**: Evaluates both conditions for `and`.
- **Step-by-Step Evaluation**:
  - If `cpu_usage` is 70 and `memory_usage` is 80: **True**
  - If `cpu_usage` is 85 and `memory_usage` is 90: **False**.

14. Loan Approval Process

Expression:

```python
loan_approved = (credit_score >= 650 and income > 40000) or income > 60000
```

- **Logical Flow**: Determines if a loan can be approved.
- **Short-Circuit Evaluation**: If `credit_score >= 650` is `False`, `income > 40000` is not evaluated.

- **Step-by-Step Evaluation**:
  - If `credit_score` is 700 and `income` is 45000: **True**
  - If `credit_score` is 600 and `income` is 65000: **True**.

15. Insurance Eligibility

Expression:

```python
eligible_for_insurance = (age < 50 or medical_checkup) and age > 30
```

- **Logical Flow**: Checks if a person is eligible for insurance.
- **Short-Circuit Evaluation**: If `age < 50` is `True`, `medical_checkup` is not evaluated.
- **Step-by-Step Evaluation**:
  - If `age` is 45 and `medical_checkup` is `False`: **True**
  - If `age` is 55: **False**.

16. System Maintenance Window

Expression:

```python
schedule_maintenance = (is_weekend or low_traffic) and not low_traffic
```

- **Logical Flow**: Determines if system maintenance can be scheduled.
- **Short-Circuit Evaluation**: If `is_weekend` is `True`, `low_traffic` is not evaluated.
- **Step-by-Step Evaluation**:
  - If `is_weekend` is `True` and `low_traffic` is `False`: **False**
  - If `is_weekend` is `False` and `low_traffic` is `True`: **True**.

17. Multi-Factor Authentication

Expression:

```python
mfa_successful = password_correct and (otp_verified or not otp_verified
```

- **Logical Flow**: Checks if multi-factor authentication succeeds.
- **Short-Circuit Evaluation**: If `password_correct` is `False`, `(otp_verified or not otp_verified)` is not evaluated.
- **Step-by-Step Evaluation**:
  - If `password_correct` is `True` and `otp_verified` is `True`: **True**
  - If `password_correct` is `False`: **False**.

18. Job Promotion Eligibility

Expression:

```python
promotion_approved = (experience_years > 3 or has_recommendation) and not has_recommendation
```

- **Logical Flow**: Checks if an employee is eligible for a promotion.
- **Short-Circuit Evaluation**: If `experience_years > 3` is `True`, `has_recommendation` is not evaluated.
- **Step-by-Step Evaluation**:
  - If `experience_years` is 5 and `has_recommendation` is `False`: **True**
  - If `experience_years` is 2 and `has_recommendation` is `True`: **False**.

19. Customer Support Chat Availability

Expression:

```python
can_join_chat = agents_available or (queue_length < 10 and not agents_available)
```

- **Logical Flow**: Determines if a customer can join a support chat.
- **Short-Circuit Evaluation**: If `agents_available` is `True`, `(queue_length < 10 and not agents_available)` is not evaluated.
- **Step-by-Step Evaluation**:
  - If `agents_available` is `True`: **True**
  - If `agents_available` is `False` and `queue_length` is 5: **True**.

20. E-Commerce Cart Validation

Expression:

```python
checkout_possible = (items_in_cart > 0 and logged_in) or (not logged_in and items_in_cart > 2)
```

- **Logical Flow**: Checks if a customer can proceed to checkout.
- **Short-Circuit Evaluation**: If `items_in_cart > 0` is `True`, `logged_in` is not evaluated.
- **Step-by-Step Evaluation**:
  - If `items_in_cart` is 4 and `logged_in` is `False`: **True**
  - If `items_in_cart` is 1 and `logged_in` is `False`: **False**.

Conclusion

This detailed analysis covers all 20 logical expressions, explaining the logical flow, short-circuit evaluation, and step-by-step reasoning for each scenario. You can use this framework to understand how to evaluate logical expressions in programming effectively.

Share Rewrite

---

tell me this pdf too all questions

Sources

Sheet of 20 Complex Expressions (With Variables and Problem Context)
File · 1

Set of 20 Logical Expressions (Short-Circuit Evaluation)
File · 2

Answer

Here's a comprehensive breakdown of all 20 questions from the PDF titled "Basic Expressions" along with their evaluations and explanations.

1. Temperature Conversion

Expression:

```python
celsius = 30
fahrenheit = (9 / 5) * celsius + 32 - 5 / 2
```

Question: Convert Celsius to Fahrenheit and evaluate the expression.

Evaluation:

- $\text{fahrenheit} = \left(\frac{9}{5} \times 30\right) + 32 - \frac{5}{2}$
- $\text{fahrenheit} = 54 + 32 - 2.5 = 83.5$

Result: 83.5 Fahrenheit

2. Sales Calculation

Expression:

```python
price = 150
discount = 20
tax = 5
final_price = price - (price * discount / 100) + tax ** 2 // 3
```

Question: Calculate the final_price after applying discount and tax.

Evaluation:

- $\text{final price} = 150 - 30 + \frac{25}{3} \approx 150 - 30 + 8.33 \approx 128.33$

Result: 128.33

3. Area of a Complex Shape

Expression:

```python
radius = 7
side = 4
area = (3.14 * radius ** 2) + (side ** 2 - 2 * side + 5 % 2)
```

Question: Calculate the area of a complex shape (circle + modified square).

Evaluation:

- $\text{area} = (3.14 \times 7^2) + (4^2 - 2 \times 4 + 5\%2)$
- $\text{area} = (3.14 \times 49) + (16 - 8 + 1) = 153.86 + 9 = 162.86$

Result: 162.86

4. Monthly Budget Calculation

Expression:

```python
income = 5000
rent = 1200
groceries = 300
savings = income - (rent + groceries * 2) // 3 + (savings_percentage := 10)
```

Question: Compute the savings from the monthly income after rent and groceries with a 10% savings rate.

Evaluation:

- $\text{savings} = 5000 - (1200 + 300 \times 2) //3 + 10$
- $\text{savings} = 5000 - (1200 + 600)//3 + 10 = 5000 - 1800//3 + 10 = 5000 - 600 + 10 = 4410$

Result: 4410

5. Profit/Loss Calculation

Expression:

```python
cost_price = 200
selling_price = 180
profit_loss = (selling_price - cost_price) * 2 + cost_price // 5 - 3 ** 2
```

Question: Find the total profit or loss from the selling price and cost price.

Evaluation:

- $\text{profit loss} = (180 - 200) \times 2 + 200//5 - 9$
- $\text{profit loss} = -20 \times 2 + 40 - 9 = -40 + 40 - 9 = -9$

Result: -9 (Loss)

6. Physics Problem - Velocity

Expression:

```python
distance = 100
time = 5
velocity = distance / time + 2 ** 2 - 5 * 3 // 2
```

Question: Calculate the velocity of a moving object.

Evaluation:

- $\text{velocity} = \frac{100}{5} + 2^2 - \frac{15}{2}$
- $\text{velocity} = 20 + 4 - 7.5 = 16.5$

Result: 16.5 m/s

7. Inventory Calculation

Expression:

```python
initial_stock = 500
sold_units = 123
restock = 50
current_stock = initial_stock - sold_units + restock ** 2 // 10 + 7 % 3
```

Question: Compute the current_stock after sold units and restock.

Evaluation:

- $\text{current stock} = 500 - 123 + \frac{50^2}{10} + 7\%3$
- $\text{current stock} = 500 - 123 + 250 + 1 = 628$

Result: 628

8. Exam Marks Calculation

Expression:

```python
total_marks = 500
marks_obtained = 430
bonus = 5
percentage = (marks_obtained + bonus) * 100 / total_marks - 3 // 2
```

Question: Calculate the student's percentage including a bonus of 5 marks.

Evaluation:

- $\text{percentage} = \frac{(430+5)\times 100}{500} - 1$
- $\text{percentage} = \frac{43500}{500} - 1 = 87 - 1 = 86$

Result: 86%

9. Compound Interest

Expression:

```python
principal = 1000
rate = 5
time = 2
interest = principal * (1 + rate / 100) ** time - 5 * 3 + 7 % 2
```

Question: Compute the compound interest using the given formula with adjustments.

Evaluation:

- $\text{interest} = 1000 \times (1 + 0.05)^2 - 15 + 1$
- $\text{interest} = 1000 \times 1.1025 - 15 + 1 = 1102.5 - 15 + 1 = 1088.5$

Result: 1088.5

10. Acceleration of a Car

Expression:

```python
initial_velocity = 10
final_velocity = 50
time = 5
```

```
acceleration = (final_velocity - initial_velocity) / time + 5 % 3 * 4
```

Question: Calculate the acceleration of the car.

Evaluation:

- $\text{acceleration} = \frac{50-10}{5} + 2 \times 4$
- $\text{acceleration} = \frac{40}{5} + 8 = 8 + 8 = 16$

Result: 16 m/s²

11. GPA Calculation

Expression:

```python
credits = 18
points = 75
gpa = (points / credits) + (extra_credit := 2) ** 2 // 3 + 10 % 3
```

Question: Find the GPA with an extra credit adjustment.

Evaluation:

- $\text{gpa} = \frac{75}{18} + \frac{2^2}{3} + 1$
- $\text{gpa} = 4.1667 + 1.3333 + 1 = 6.5$

Result: 6.5

12. Employee Salary Calculation

Expression:

```python
base_salary = 2000
overtime = 5
deductions = 100
final_salary = base_salary + (overtime * 20) - deductions // 2 + 10 % 4
```

Question: Calculate the final_salary after overtime and deductions.

Evaluation:

- $\text{final salary} = 2000 + (5 \times 20) - \frac{100}{2} + 2$

- final salary $= 2000 + 100 - 50 + 2 = 2052$

Result: 2052

13. Shopping Bill Calculation

Expression:

```python
items = 10
item_price = 30
discount = 15
total = (items * item_price - discount) + discount // 3 * 2
```

Question: Compute the total shopping bill after applying the discount.

Evaluation:

- total $= (10 \times 30 - 15) + \frac{15}{3} \times 2$
- total $= (300 - 15) + 10 = 295 + 10 = 305$

Result: 305

14. Distance Between Two Points

Expression:

```python
x1 = 3
y1 = 4
x2 = 6
y2 = 8
distance = ((x2 - x1) ** 2 + (y2 - y1) ** 2) ** 0.5 + 5 // 2
```

Question: Find the distance between two points on a plane.

Evaluation:

- distance $= \sqrt{(6-3)^2 + (8-4)^2} + 2$
- distance $= \sqrt{3^2 + 4^2} + 2 = \sqrt{9 + 16} + 2 = \sqrt{25} + 2 = 5 + 2 = 7$

Result: 7

15. Work Efficiency Calculation

Expression:

```python
total_work = 100
work_done = 70
efficiency = (work_done / total_work) * 100 + 5 * 2 // 3
```

Question: Compute the work efficiency in percentage.

Evaluation:

- $\text{efficiency} = \left(\frac{70}{100}\right) \times 100 + \frac{10}{3}$
- $\text{efficiency} = 70 + 3.33 \approx 73.33$

Result: 73.33%

16. Electricity Bill Calculation

Expression:

```python
units = 250
rate = 5
surcharge = 50
bill = (units * rate) + surcharge // 2 * 3 + 7 % 4
```

Question: Calculate the total electricity bill after adding the surcharge.

Evaluation:

- $\text{bill} = (250 \times 5) + \frac{50}{2} \times 3 + 3$
- $\text{bill} = 1250 + 75 + 3 = 1328$

Result: 1328

17. Body Mass Index (BMI) Calculation

Expression:

```python
```

```
weight = 70  # kg
height = 1.75  # meters
bmi = weight / height ** 2 + 10 % 3 * 2 // 1
```

Question: Compute the BMI of a person and adjust using the remainder operator.

Evaluation:

- $\text{bmi} = \frac{70}{1.75^2} + 2$
- $\text{bmi} = \frac{70}{3.0625} + 2 \approx 22.86 + 2 = 24.86$

Result: 24.86

18. Loan Payment Calculation

Expression:

```
python
```

```
loan_amount = 5000
interest_rate = 7
time = 3
emi = (loan_amount * interest_rate / 100 * time) // 12 + 5 % 2 * 3
```

Question: Calculate the EMI (equated monthly installment) for the loan.

Evaluation:

- $\text{emi} = \left(\frac{5000 \times 7 \times 3}{100}\right) //12 + 1 \times 3$
- $\text{emi} = 1050//12 + 3 = 87.5 + 3 = 90.5$

Result: 90.5

19. Savings Growth Calculation

Expression:

```
python
```

```
savings = 2000
growth_rate = 10
years = 5
growth = savings * (1 + growth_rate / 100) ** years // 2 + 7 % 3
```

Question: Calculate the growth in savings over a period of 5 years.

Evaluation:

- $\text{growth} = 2000 \times (1 + 0.1)^5 // 2 + 1$
- $\text{growth} = 2000 \times 1.61051 // 2 + 1 \approx 3221.02 // 2 + 1 \approx 1610.51 + 1 = 1611.51$

Result: 1611.51

20. Running Race - Time to Finish

Expression:

```python
distance = 100  # meters
speed = 8  # meters/second
time_to_finish = distance / speed + 10 % 2 * 3 // 1
```

Question: Compute the time taken to finish the race.

Evaluation:

- $\text{time to finish} = \frac{100}{8} + 0$
- $\text{time to finish} = 12.5 + 0 = 12.5$

Result: 12.5 seconds This comprehensive breakdown covers all 20 questions from the PDF, providing evaluations and results for each expression. You can use this structure to understand how to approach similar problems in the future.

Related

How do these expressions apply to real-world scenarios +
What are some common mistakes when solving these expressions +
How can I optimize the calculation of these expressions +
What is the significance of operator precedence in these expressions +
How do these expressions relate to problem-solving in physics or engineering +